DOCUMENTATION

ASSIGNMENT *2*

STUDENT NAME: Tianu Cosmin-Nicolae
GROUP: 30425

CONTENTS

1. Assignment Objective

Design and implement an application aiming to analyze queuing-based systems by (1) simulating a series of N clients arriving for service, entering Q queues, waiting, being served and finally leaving the queues, and (2) computing the average waiting time, average service time and peak hour.

I. ***Analyze the problem and identify requirements.***

II. ***Design the simulation application.***

III. ***Implement the simulation application.***

IV. ***Test the simulation application.***

2. Problem Analysis, Modeling, Scenarios, Use Cases

The queue management application has the following ***functional requirements****. It should enable the user to start the simulation, and closely observe the evolution of the simulation. Also, the user has the ability to set the simulation parameters, such as the* ***number of clients (N) and of queues (Q), the maximum simulation time*** *and* ***the bounds of the client parameters: arrival time and service time,*** *through the application. Moreover, at the end of the simulation the app should inform the user about* ***the average waiting time, average service time and the peak hour*** *of the simulation. Finally, all the app needs to log all the simulation information in a .txt file.*

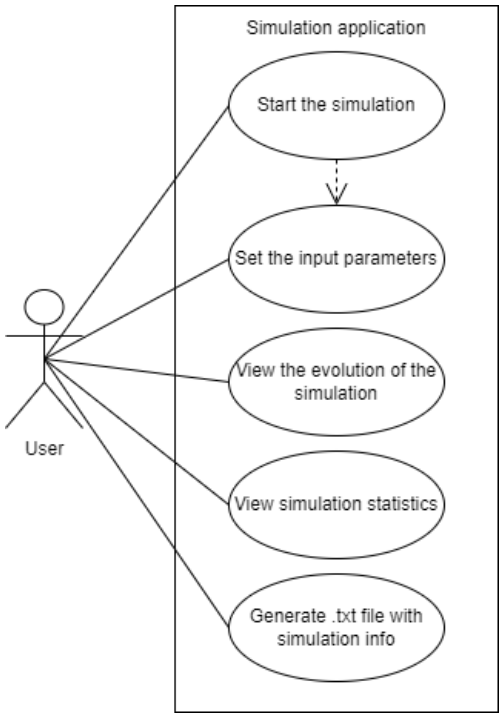

Figure 1: Use case diagram.

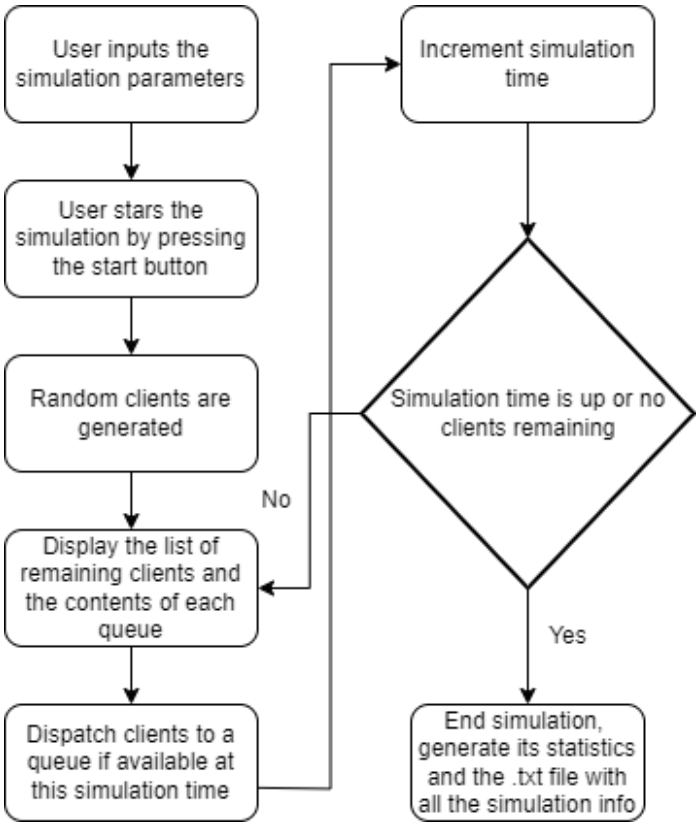

Figure 2: Flow-chart of the application.

*As for the **non-functional requirements** the queue management app should be easy and intuitive to use by the user.*

3. Design

*The queue management app's structure follows a **MVC structure**. The **model** of the app would represent the core functionality and data handling, therefore here are present the **Task** and **Server** classes and also, the **SelectionPolicy** enumeration. The **Server class** represents one queue that are usually found in supermarkets or every other store, the **Task class** symbolizes a person that wants to purchase anything from that store and needs to pay, while the **SelectionPolicy** is just an enumeration that contains the **strategy** that the app could use .The **view** handles the presentation of the user interface, it also contains graphical elements such as buttons and text fields. Additionally, it has only one class: **SimulationFrame** that incorporates logic for sending the user input to the controller and updating the GUI with the simulation info at each time of the simulation. The **controller** acts as the intermediary between the model and view, it takes the given input from the view, processes it, sends each task to the appropriate time using the **SimulationManager**, and to the appropriate server through the **Scheduler** using one of the possible **Strategies**. Its job is to coordinate user input, trigger events and update the view along the way.*

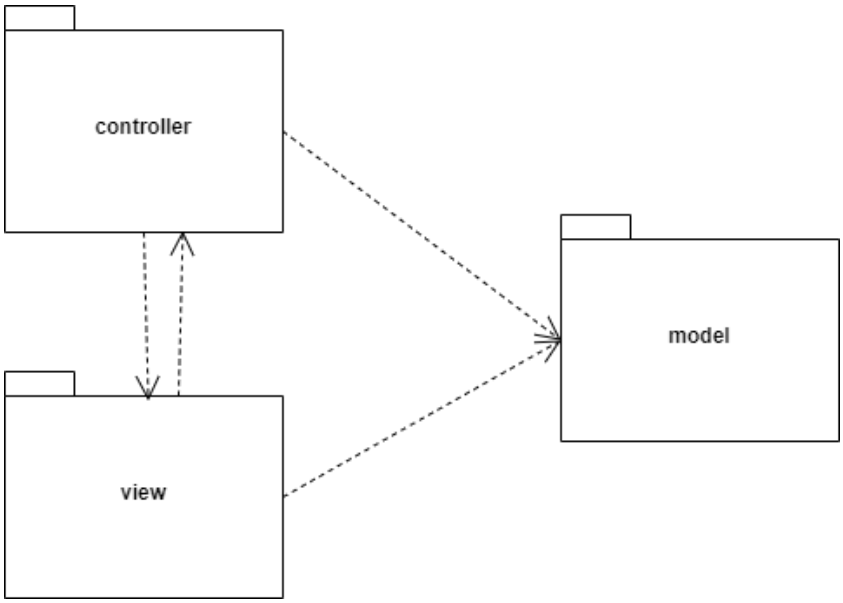

Figure 3: UML package diagram

4. Implementation

*Starting with **model package** it contains the **Task** class, as already mentioned it represents a person. Therefore, each person has an **id**, an **arrival time** (the time the person is ready for check out) and the **service time** (the time necessary for the person to finish the paying process). An important thing to mention is the **overriding** of the **toString method** for an easier understanding of the output or debugging.*

```
@Override
public String toString() {
    return "(" + id + "," + arrivalTime + "," + serviceTime + ")";
}
```

Figure 4: toString method

*As for the **Server** class, it represents a queue where a task can be processed and works on a separate thread, thus implementing the **Runnable** interface. Since the main and gui updating thread can access the contents of the server class at the same time, the **tasks** field which represents the current tasks in the queue is of type **BlockingQueue** and the **waiting period** is of type **AtomicInteger**, these types are known and were created to support multi-threading in a safe way. Moreover, the same applies to the field **running** which is an **AtomicBoolean** which is used as the stop condition for the servers' threads. Finally, the **totalWaitingTime** field stores the amount of time that each task has spent in the queue until its time to be processed started, using*

*the **currentTime** (which is passed down from the main thread in the **SimulationManager**, through the **Scheduler**) and the task's arrival time.*

```java
@Override
public void run() {
    int prevTaskId = 0;
    while (running.get() && !Thread.currentThread().isInterrupted()) {
        try {
            Task task = tasks.peek();

            if (task != null) {
                int currentTaskId = task.getId();

                if(prevTaskId != currentTaskId){
                    totalWaitingTime += currentTime.get() - task.getArrivalTime();
                    System.out.println("The task waited needs to wait for : " + totalWaitingTime);
                }
                System.out.println("the curr time in the task is " + currentTime);
                prevTaskId = currentTaskId;

                System.out.println("Task " + task.getId() + " has remaining time: " + task.getServiceTime());
                TimeUnit.SECONDS.sleep( timeout: 1);
                waitingPeriod.decrementAndGet();

                task.setServiceTime(task.getServiceTime() - 1);

                if (task.getServiceTime() == 0) {
                    tasks.poll();
                    System.out.println("Task " + task.getId() + " completed.");
                }
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            System.err.println("Error processing task: " + e.getMessage());
        }
    }
}
```

Figure 5: run method of the Server Class

*The picture above shows the run method of the **Server class** that checks if there is any task in the queue, if there is it sleeps for 1 second to sync with the main thread, updates the time of the current task to match the new time. Also, if the time of a task is up, it is eliminated from the list of tasks of the server and if a new task arrives at the front of the queue if calcultes the time it has waited in the queue until then.*

*Finally, the last class of the **model** package is the **SelectionPolicy enum** which stores the available strategies of the queue management app.*

```
public enum SelectionPolicy {
    2 usages
    SHORTEST_QUEUE,
    2 usages
    SHORTEST_TIME
}
```

Figure 6: SelectionPolicy enum

*The **view package** has only one class, the **simulationFrame** class which extends **Jframe** and it represent the **GUI** thus, using Swing the user can communicate with the logic of the application. This **JFrame** consists of the input panel that has **JLabels** and **JTextFields** that allows the user to input the following: the maximum **simulation time**, **the number of servers**, the number of tasks, the **minimum** and **maximum** time that a task can **arrive** at, the **minimum** and **maximum** time that a task can be **processed** in and also a **JComboBox** that enables the user to switch between the queue management **strategies**. Also, it contains a **JButton** that has the scope to start the simulation, thus sending the data from the GUI to the **SimulationManager** through an **ActionListener**. Finally, the important methods here are: **updateSimulationInfo** which is called at every time of the simulation in the main thread and appends the current time information to the past simulation information, **appendStats** that is called at the end of the simulation to append the statistics and **writeContentsToFile which** has the role to take everything that has been output (the whole simulation information and it's statistics) and write them intro a .txt file.*

```
public  void updateSimulationInfo(String info) {
    textArea.append(info + "\n");
    textArea.setCaretPosition(textArea.getDocument().getLength());
}
```

Figure 7: updateSimulationInfo method

```
public  void appendStats(int peakTime, double averageWaitingTime, double averageServiceTime){
    textArea.append("END OF SIMULATION \n");
    textArea.append("Peak time : " + peakTime + "\n");
    textArea.append("Average waiting time : " + averageWaitingTime + "\n");
    textArea.append("Average service time : " + averageServiceTime + "\n");
    textArea.setCaretPosition(textArea.getDocument().getLength());
}
```

Figure 8: appendStats method

```java
public void writeContentsToFile(String fileName) {
    try (BufferedWriter writer = new BufferedWriter(new FileWriter(fileName))) {
        writer.write(textArea.getText());
        writer.flush();
        System.out.println("Contents written to file: " + fileName);
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

Figure 9: writeContentsToFile method

*As for the **controller package**, it contains the core functionality of the queue management app. This functionality resides in the **SimulationManager** class. In this class also resides the **main** method but it just creates an object of type SimulationManager. This class implements besides **Runnable** also the **SimulationManagerListener** interface that has just one method: **startSimulation**, its implementation **sets the user's input** as the simulation parameters, it creates a **scheduler** object, generates the **random tasks**, computes the **average service time** and **starts** the main, **all** the server and the GUI **threads**.*

```java
@Override
public void startSimulation(int maxSimulationTime, int numServers, int numTasks, int minArrivalTime,
                            int maxArrivalTime, int minServiceTime, int maxServiceTime,
                            SelectionPolicy selectionPolicy) {

    this.timeLimit = maxSimulationTime;
    this.maxArrivalTime = maxArrivalTime;
    this.minArrivalTime = minArrivalTime;
    this.maxServiceTime = maxServiceTime;
    this.minServiceTime = minServiceTime;
    this.numbersOfServers = numServers;
    this.numberOfClients = numTasks;

    this.scheduler = new Scheduler(numbersOfServers, selectionPolicy, currentTime);

    generateNRandomTasks(this.numberOfClients, this.minArrivalTime, this.maxArrivalTime,
            this.minServiceTime, this.maxServiceTime);

    averageServiceTime = computeAverageServiceTime();

    Thread simulationManagerThread = new Thread( task: this);

    simulationManagerThread.start();
    scheduler.startThreads();
    startGUIThread();

}
```

Figure 10: startSimulation method

*The **main thread** only sleeps for **1** seconds and increments the current time, simulating a real time queue. It also **sends** any **available tasks** at the current time to a server according to the selected startetgy, **computes the peak time** by checking the number of tasks at each time and **exits the simulation** if **the time is u**p or if there are **no more tasks coming up or being processed**. After the simulation is over the statistics are added to the GUI, the simulation info is written in the .txt file and the gui and sever threads are **stopped**.*

```java
@Override
public void run() {
    try {
        while (currentTime.get() <= timeLimit) {
            System.out.println("Time : " + currentTime);

            // Process tasks arrival
            processArrivalTasks();

            computePeakTime(currentTime);

            // Sleep for 1 second to simulate time passing
            TimeUnit.SECONDS.sleep( timeout: 1);

            // Increment the current time
            currentTime.incrementAndGet();

            // If no more task to be or being processed, stop simulation
            if (!checkIfAnyTasksRemained()) {
                break;
            }
        }
        stopGUIThread();

        simulationFrame.appendStats(peakTime, computeAverageWaitingTime(), averageServiceTime);
        simulationFrame.writeContentsToFile(logFileName);

    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    } finally {
        // Stop servers
        scheduler.stopServers();
    }
}
```

Figure 11: run method of the SimulationManager class

The statistics are calculted using their own methods :

```
public synchronized void computePeakTime(AtomicInteger currentTime) {
    int sumOfClients = 0;
    for (Server server : scheduler.getServers()) {
        sumOfClients += server.getTasks().size();
    }
    System.out.println("the nr of clients : " + sumOfClients);
    if (sumOfClients > maxNumClientsAtCurrentTime) {
        maxNumClientsAtCurrentTime = sumOfClients;
        peakTime = currentTime.get();
    }
}
public double computeAverageWaitingTime() {
    int sumOfWaitingTimes = 0;
    for (Server server : scheduler.getServers()) {
        sumOfWaitingTimes += server.getTotalWaitingTime();
    }
    return (double) sumOfWaitingTimes / numberOfClients;
}
public double computeAverageServiceTime() {
    int totalServiceTime = 0;
    for (Task task : generatedTasks) {
        totalServiceTime += task.getServiceTime();
    }
    return (double) totalServiceTime / generatedTasks.size();
}
```

Figure 12: statistics methods

*The last thing to be noted about this class is the way the **GUI thread** works. Basically it uses the **updateGUI** then waits for 1 second to be in sync with the main thread until it is stopped using a flag **guiThreadRunning.***

```java
private void startGUIThread() {
    Thread guiThread = new Thread(() -> {
        while (guiThreadRunning) {
            try {
                SwingUtilities.invokeLater(this::updateGUI);
                TimeUnit.SECONDS.sleep( timeout: 1);

            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    });
    guiThread.start();
}
```

```java
private void stopGUIThread() {
    guiThreadRunning = false;
}
```

```java
private void updateGUI() {

    StringBuilder info = new StringBuilder();
    info.append("Time: ").append(currentTime).append("\n");
    info.append("Waiting clients:").append(generatedTasks).append("\n");

    for (int i = 0; i < scheduler.getServers().size(); i++) {
        BlockingQueue<Task> tasks = scheduler.getServers().get(i).getTasks();
        info.append("Queue ").append(i + 1).append(": ");
        if (tasks.isEmpty()) {
            info.append("closed");
        }
        for (Task task : tasks) {
            info.append("(").append(task.getId()).append(",").append(task.getArrivalTime()).append(",")
                    .append(task.getServiceTime()).append("); ");
        }
        info.append("\n");
    }

    simulationFrame.updateSimulationInfo(info.toString());
}
```

Figure 13: GUI Thread operation methods

*The second most significant class of this package is the **Scheduler**. In this class the strategy is set and the tasks are dispacthed also here according to it. More importantly, in its constructor the **Server** objects and their **Threads** are created and added to a list for ease of use.*

```
public Scheduler(int nrServers,SelectionPolicy selectionPolicy, AtomicInteger currentTime) {

    changeStrategy(selectionPolicy);

    servers = new ArrayList<>();
    threads = new ArrayList<>();
    for (int i = 0; i < nrServers; i++) {
        Server server = new Server(currentTime);
        Thread serverThread = new Thread(server);
        threads.add(serverThread);
        servers.add(server);
    }
}
```

Figure 14: GUI Thread operation methods

The rest of the controller package consists of the 2 strategies: **ShortestQueueStrategy** and **ShortestTimeStrategy** classes that both implement the **Strategy** interface.

```
@Override
public void addTask(List<Server> servers, Task task) {
    int minWaitingTime = servers.getFirst().getWaitingPeriod().get();
    int minWaitingTimeIndex = 0;
    for(int i = 0; i < servers.size();i++){
        if(servers.get(i).getWaitingPeriod().get() < minWaitingTime){
            minWaitingTime=servers.get(i).getWaitingPeriod().get();
            minWaitingTimeIndex = i;
        }
    }
    servers.get(minWaitingTimeIndex).addTask(task);
}
```

Figure 15: ShortestTimeStrategy addTask method

```java
@Override
public void addTask(List<Server> servers, Task task) {
    int minElemSize = servers.getFirst().getTasks().size();
    int minElemSizeIndex = 0;
    for (int i = 0; i < servers.size(); i++) {
        if (servers.get(i).getTasks().size() < minElemSize) {
            minElemSize = servers.get(i).getTasks().size();
            minElemSizeIndex = i;
        }
    }
    servers.get(minElemSizeIndex).addTask(task);
}
```

Figure 16: ShortestQueueStrategy addTask method

```java
public interface Strategy {
    1 usage  2 implementations  cosmintianu
    void addTask(List<Server> servers, Task task);
}
```

Figure 17: Strategy interface

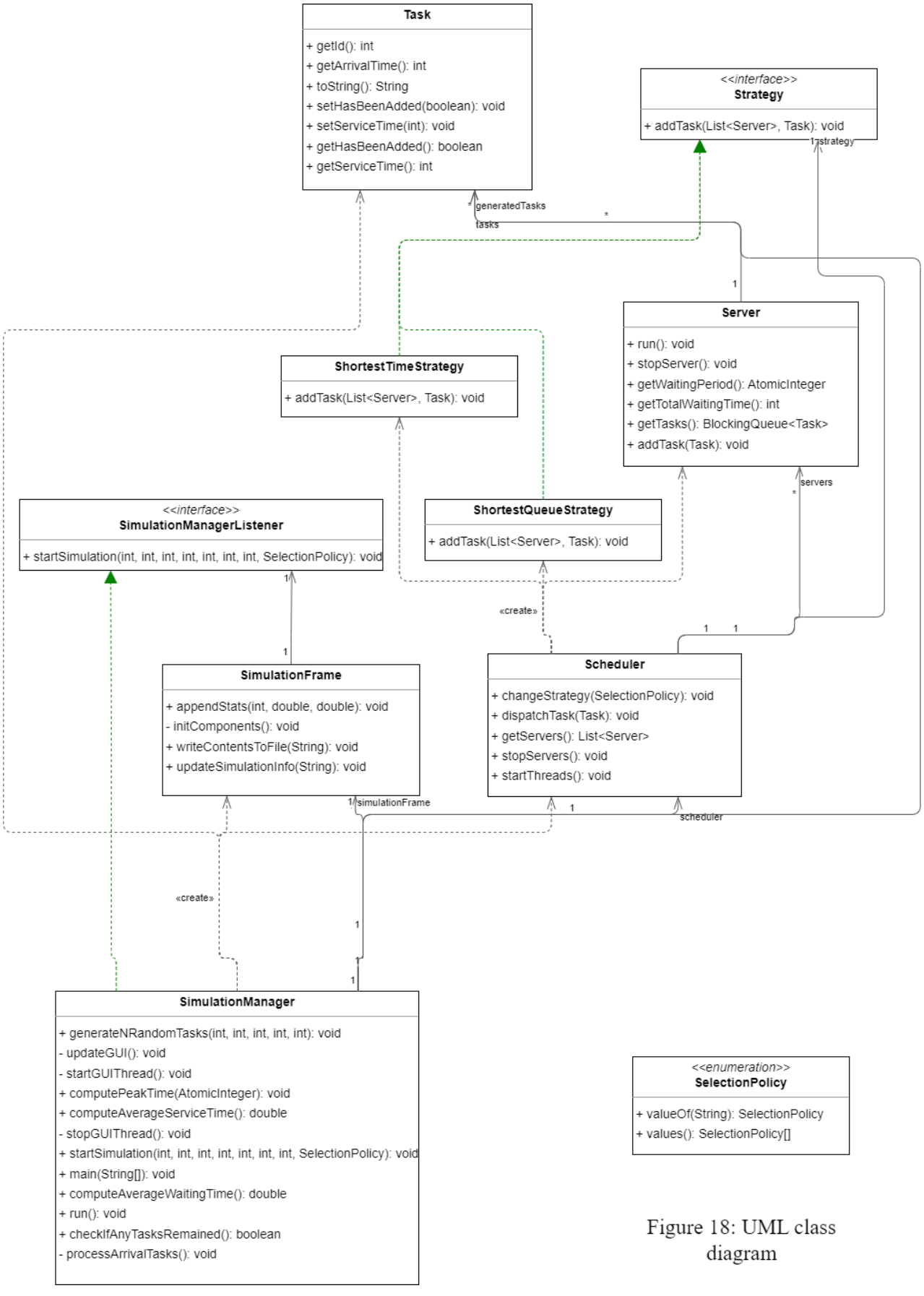

Figure 18: UML class diagram

5. Conclusions

*In **conclusion**, a queue management system exists at almost supermarket we go to and it is interesting to know how one works. Personally, I learned to use Threads, to create a multi-thread environment, synchronize them and use them in the most efficient way, also this opportunity provided me with extra coding experience and a new project in my portfolio.*

*For **further developments**, the GUI could use a make over.*

6. Bibliography

1. *https://dsrl.eu/courses/pt/materials/PT2024_A2.pdf*
2. *https://dsrl.eu/courses/pt/materials/PT_2024_A2_S1.pdf*
3. *https://dsrl.eu/courses/pt/materials/PT_2024_A2_S2.pdf*
4. *https://www.geeksforgeeks.org/multithreading-in-java/*
5. *https://www.geeksforgeeks.org/synchronization-in-java/*